

EduOS AI — Course Curriculum

Full-Stack Programming Bootcamp · Academic Year 2024 · 12 Weeks · Batch A & B

Topic 1: Arrays (Week 1)

Prerequisites	None
Difficulty	Beginner
Estimated Hours	8

Introduction to arrays, indexing, traversal, insertion, deletion, and common patterns like two-pointer and sliding window.

Learning Outcomes:

- Declare and initialize arrays in multiple languages
- Traverse arrays using loops and iterators
- Implement two-pointer technique for pair problems
- Apply sliding window for subarray problems

Topic 2: Linked Lists (Week 2)

Prerequisites	Arrays
Difficulty	Beginner-Intermediate
Estimated Hours	10

Singly and doubly linked lists, node structure, pointer manipulation, reversal, cycle detection (Floyd's algorithm), and merge operations.

Learning Outcomes:

- Implement a singly linked list from scratch
- Reverse a linked list iteratively and recursively
- Detect cycles using Floyd's tortoise-and-hare
- Merge two sorted linked lists

Topic 3: Recursion (Week 3)

Prerequisites	Arrays, Functions
Difficulty	Intermediate (High Drop Risk)
Estimated Hours	14

Fundamental recursive thinking: base cases, recursive cases, call stack visualization. Topics include factorial, Fibonacci, power functions, tree recursion, memoization basics, and tail recursion. Students must internalize the call stack before proceeding to Trees or Dynamic Programming. Common failure point: students memorize patterns without understanding how the stack unwinds. Requires deliberate practice with call-stack diagrams and tracing exercises.

Learning Outcomes:

- Identify and write correct base cases

- Trace a recursive call stack on paper
- Implement Fibonacci with and without memoization
- Convert simple iterative solutions to recursive ones
- Recognize when recursion leads to exponential time and apply memoization

[!] Instructor Note: This is the #1 topic where students disengage. Attendance drops, video completion falls, and quiz scores cluster around 40-55%. Immediate intervention recommended if a student scores below 60 on the first attempt. Do not allow students to proceed to Trees without passing Recursion.

Topic 4: Stacks and Queues (Week 4)

Prerequisites	Arrays, Linked Lists
Difficulty	Intermediate
Estimated Hours	10

Stack (LIFO) and Queue (FIFO) data structures. Applications: balanced parentheses, next greater element, BFS traversal.

Learning Outcomes:

- Implement stack and queue using arrays and linked lists
- Solve balanced parentheses problems
- Use stacks for expression evaluation
- Implement queue using two stacks

Topic 5: Trees (Week 5)

Prerequisites	Recursion, Linked Lists
Difficulty	Intermediate-Advanced
Estimated Hours	14

Binary trees, BST, tree traversals (in/pre/post/level-order), height, diameter, LCA. Requires strong recursion foundation.

Learning Outcomes:

- Implement a binary search tree with insert and search
- Perform all four tree traversals
- Calculate tree height and diameter recursively
- Find LCA in a binary tree

Topic 6: Graphs (Week 6)

Prerequisites	Trees, Stacks and Queues
Difficulty	Advanced
Estimated Hours	16

Graph representation (adjacency list/matrix), BFS, DFS, topological sort, connected components, shortest path (Dijkstra).

Learning Outcomes:

- Represent a graph using adjacency list
- Implement BFS and DFS from scratch
- Detect cycles in directed and undirected graphs
- Apply Dijkstra's algorithm for shortest path

Topic 7: Sorting Algorithms (Week 7)

Prerequisites	Arrays, Recursion
Difficulty	Intermediate
Estimated Hours	12

Bubble, selection, insertion, merge, quick sort. Time/space complexity analysis. Stable vs unstable sorts.

Learning Outcomes:

- Implement merge sort and quick sort
- Analyze time and space complexity of each algorithm
- Choose appropriate sort for a given problem
- Understand why merge sort is $O(n \log n)$ via recursion tree

Topic 8: Hashing (Week 8)

Prerequisites	Arrays
Difficulty	Intermediate
Estimated Hours	10

Hash maps, hash sets, collision resolution, common interview patterns: frequency count, anagram detection, two-sum.

Learning Outcomes:

- Explain how a hash function works
- Use hash maps for $O(1)$ lookups
- Solve two-sum in $O(n)$ using hashing
- Detect duplicates and find frequencies

Topic 9: Dynamic Programming (Week 9-10)

Prerequisites	Recursion, Arrays
Difficulty	Advanced
Estimated Hours	20

Memoization vs tabulation, 0/1 knapsack, longest common subsequence, coin change, DP on strings and grids.

Learning Outcomes:

- Convert recursive solution to memoized DP
- Build bottom-up DP tables
- Solve 0/1 knapsack and coin change
- Identify overlapping subproblems and optimal substructure

Topic 10: System Design Basics (Week 11-12)

Prerequisites	All above topics
Difficulty	Advanced
Estimated Hours	20

Scalability concepts, load balancing, caching, database design (SQL vs NoSQL), API design, CAP theorem introduction.

Learning Outcomes:

- Design a URL shortener from scratch
- Explain horizontal vs vertical scaling
- Describe when to use SQL vs NoSQL
- Design a basic REST API with proper endpoints